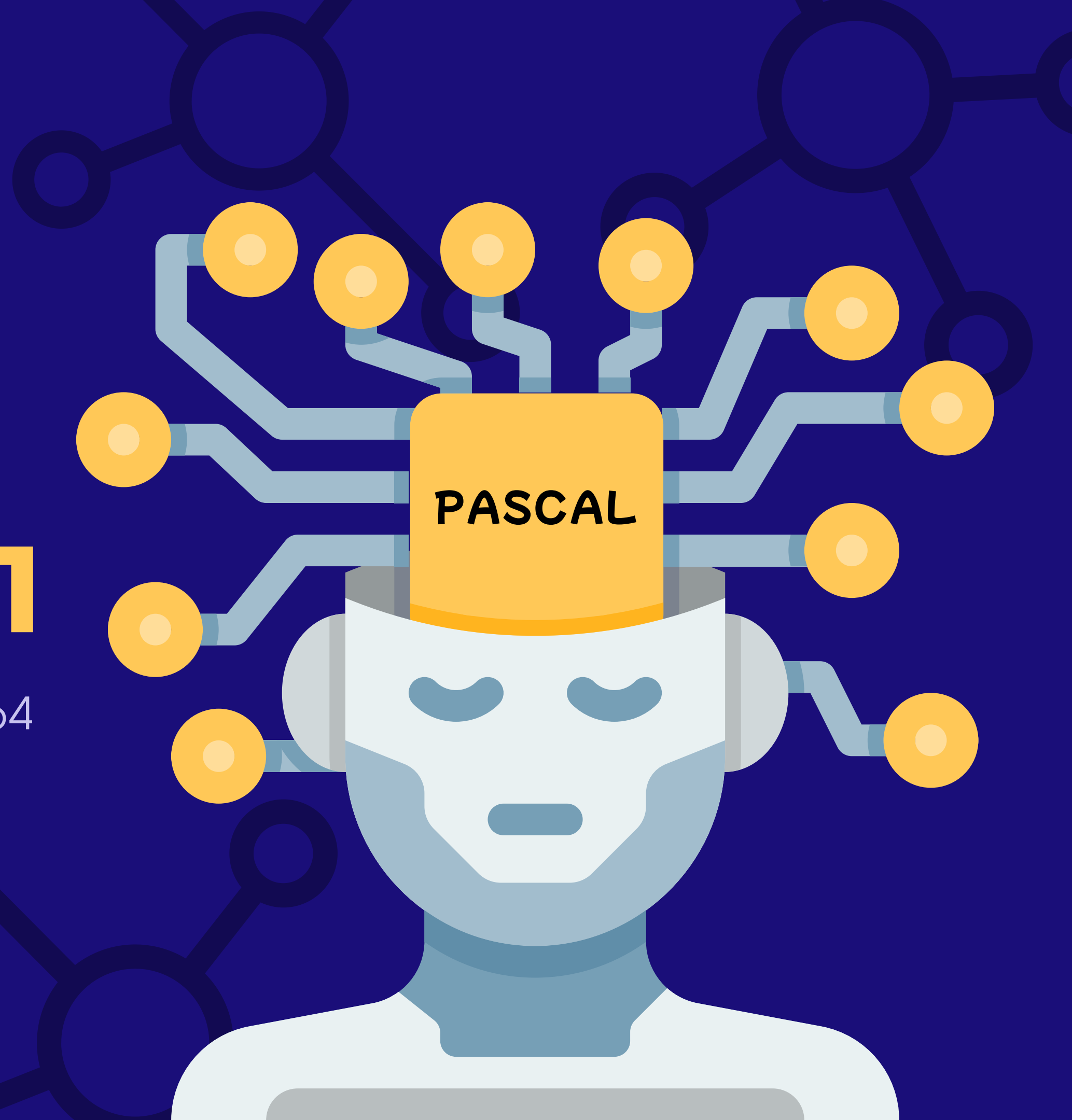


Apuntes CLASE COM260825-1

Matthew Gaviria Brenes - 2023179864

Programen en C como gente
de bien.



Diagramas

T_(orres)

Source

Target

Host



Lenguaje Host

Es muy común escribir un compilador en el mismo lenguaje en el que se implementa. Puede parecer un círculo vicioso.

C

X86

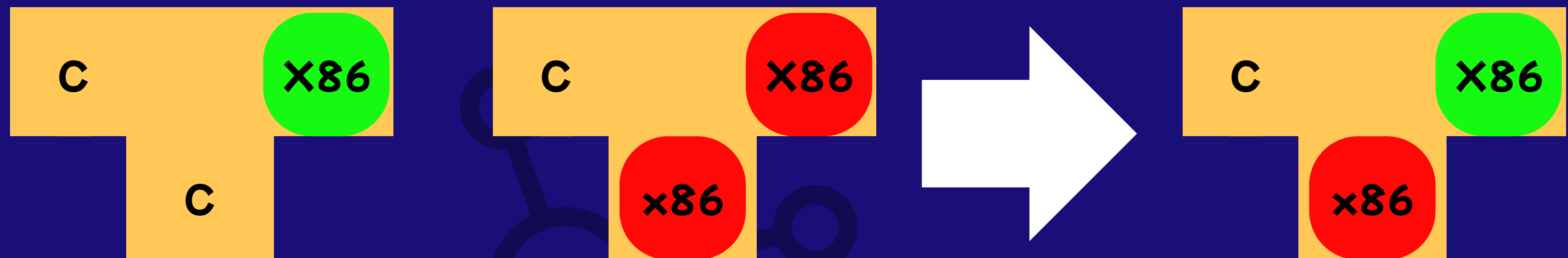
C

Compilando Compiladores

Imagine que tenemos un compilador GCC antiguo que genera código de mala calidad. También tenemos el código fuente de una versión nueva de GCC... ¿Qué hacemos? Fácil! Compilamos el compilador.

FASE 1 - COMPILANDO CON COMPILADOR ANTIGUO

Compilamos el código fuente del GCC nuevo usando el GCC viejo. Así obtenemos un nuevo GCC lento pero que genera código de calidad.

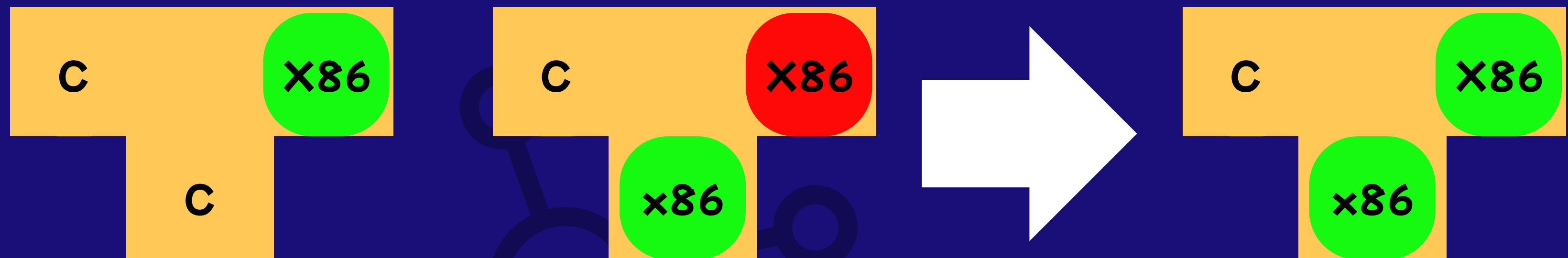


Compilando Compiladores

Imagine que tenemos un compilador GCC antiguo que genera código de mala calidad. También tenemos el código fuente de una versión nueva de GCC... ¿Qué hacemos? Fácil! Compilamos el compilador.

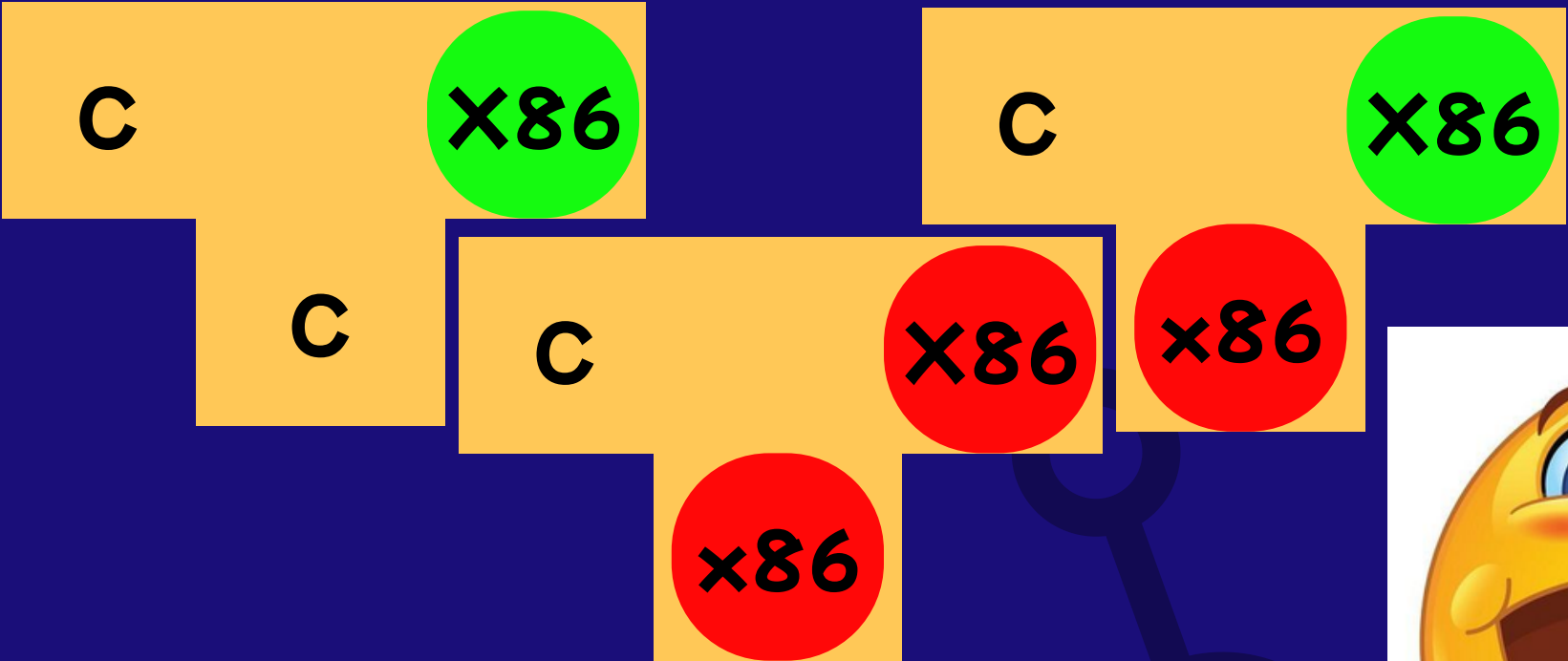
FASE 2 - COMPILANDO CON COMPILADOR NUEVO

Compilamos el código fuente del GCC nuevo otra vez, ahora usando el GCC que conseguimos en el paso anterior. Así obtenemos un nuevo GCC rápido y que genera código de calidad.

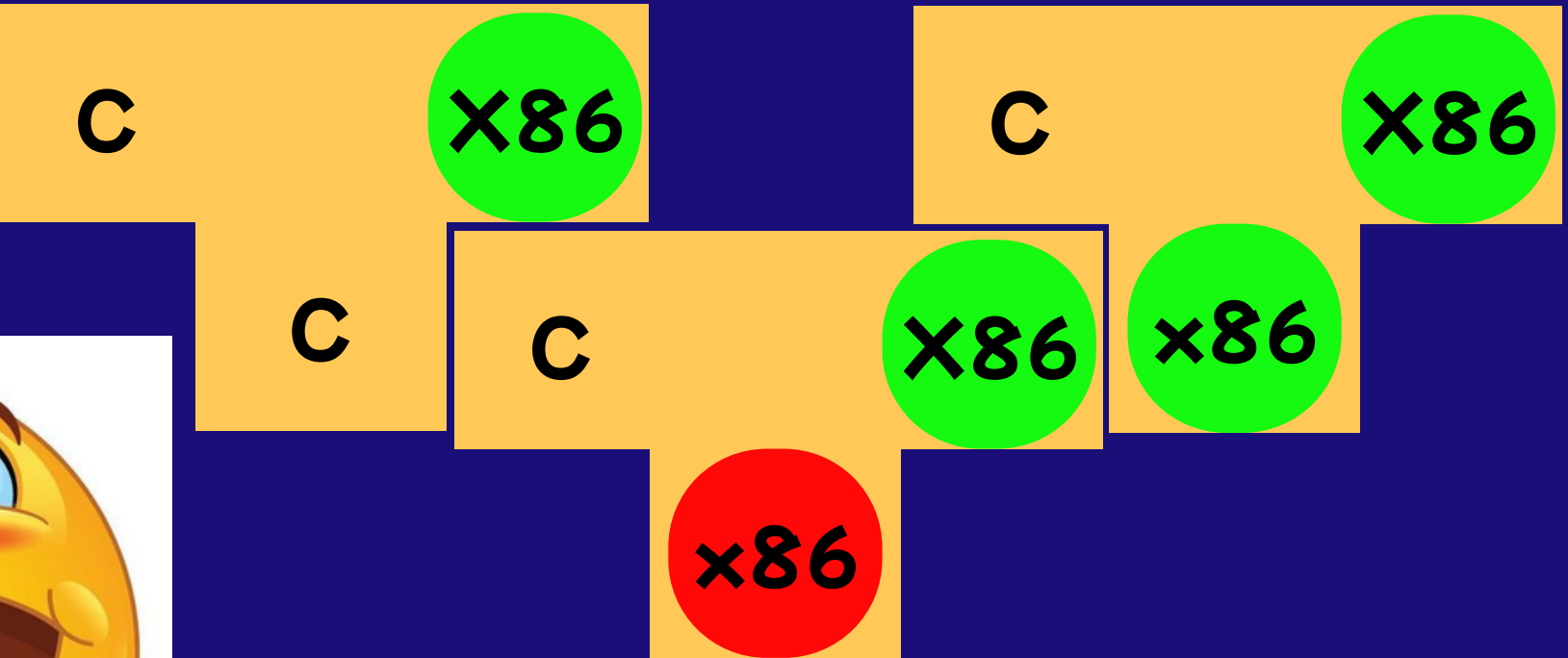


Compilando Compiladores - Diagramas T

FASE 1



FASE 2



FASE 1 Y FASE 2



Bootstrapping de Pascal

El bootstrapping consiste en usar una herramienta para construir o levantar una versión de si misma.

Es muy frecuente en Compiladores, Sistemas Operativos, Diseño de Arquitecturas, etc.

La idea central es lograr una herramienta completa usando los recursos disponibles.



Niklaus Emil Wirth

Científico de la Computación Suizo
(1934 - 2024)

**PASCAL Y P-CODE
LO MÁS DURO DEL
SISTEMA**



- Consiguió una Licenciatura en Ciencias en la ETH de Zurich
- Obtuvo una maestría en la Universidad Laval, Canadá.
- Realizó su doctorado en University of California, Berkeley.

- Establece muchos conceptos de lenguajes de programación e ingeniería de software
- Muy influyente enseñando programación y compiladores
- Diseñó lenguajes como ALGOL-W, Pascal, etc.
- Autor de “Algorithms + Data Structures = Programs”



- Ganador de un Turing Award en 1984



PASCAL

- Diseñado a principios de los años 70.
- Su nombre es un homenaje a Blaise Pascal.
- Wirth buscaba un lenguaje simple y elegante.
- La idea era fomentar buenos hábitos de programación.
- Promovía la programación estructurada.
- Pascal se volvió muy exitoso e influyente.
- Fue utilizado ampliamente en cursos introductorios de programación.
- También aparecieron versiones y derivados comerciales exitosos.

CDC6600

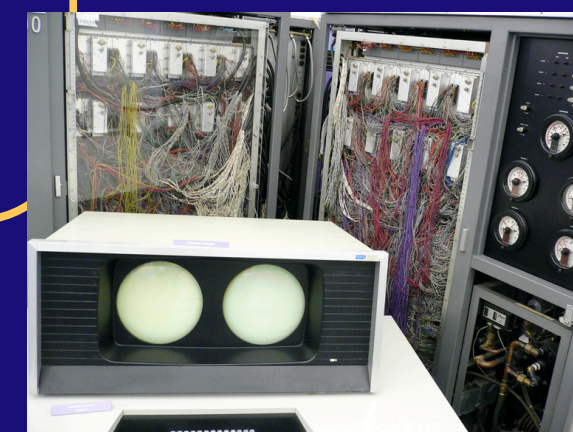
La CDC 6600 fue una de las primeras supercomputadoras. Permaneció como la computadora más rápida durante el periodo de 1964-1969 hasta que la CDC 7600 la superó. Las ideas de diseño de la CDC 6600 se mantienen vigentes en la actualidad.





PRIMER COMPILADOR DE PASCAL

- Wirth intentó inicialmente implementar Pascal utilizando FORTRAN.
- Este primer intento no resultó satisfactorio. Así que decidió escribir el compilador en el mejor lenguaje del mundo: Pascal.
- Todavía no existía un compilador de Pascal, Wirth decidió compilar el código "a mano" para que pudiera ejecutarse en la CDC 6600. De esta manera se obtuvo un compilador funcional.
- Después ocurrió el bootstrapping: El compilador podía compilar código Pascal. Incluso podía utilizarse para compilar una nueva versión de sí mismo.



Maquinas de # direcciones

Maquinas de 3 direcciones

Cada instrucción tiene 1 operador y 3 direcciones

Formato: OPERACIÓN Dir1, Dir2, Dir3

Se opera sobre Dir2 y Dir3, el resultado se guarda en Dir1.

Ejemplo:

ADD A, B, C ($A = B + C$)

DIV D, C, B ($D = C / B$)

Maquinas de 2 direcciones

Cada instrucción tiene 1 operador y 2 direcciones

Formato: OPERACIÓN Dir1, Dir2

Se opera sobre Dir1 y Dir2, el resultado se guarda en Dir1.

Ejemplo:

MOV D, C

ADD X, Y

MUL A, B

Maquinas de 1 direcciones

Cada instrucción tiene 1 operador y 1 dirección

Formato: OPERACIÓN Dir1

Se opera sobre un acumulador y Dir1, el resultado se guarda en el acumulador.

Ejemplo:

MOV D

ADD X

Maquinas de 0 direcciones

Cada instrucción tiene 1 operador y 0 direcciones.

Formato: OPERACIÓN

Opera sobre un stack. Trampa: PUSH y POP usan argumentos.

Ejemplo:

PUSH A

PUSH B

ADD

P-MACHINE (PORTABLE MACHINE)

- La P-Machine es un simulador virtual que ejecuta un código llamado P-Code.
- No depende del Hardware. Un programa compilado para esta máquina virtual puede ejecutarse en diferentes arquitecturas sin necesidad de volverse a compilar.
- Para portar el sistema basta con implementar un simulador de la P-Machine.
- Una P-Machine puede ser de 3, 2, 1 o 0 direcciones. Para una máquina sencilla, una arquitectura de 0 direcciones es la mejor opción.

Zurich P-Machine

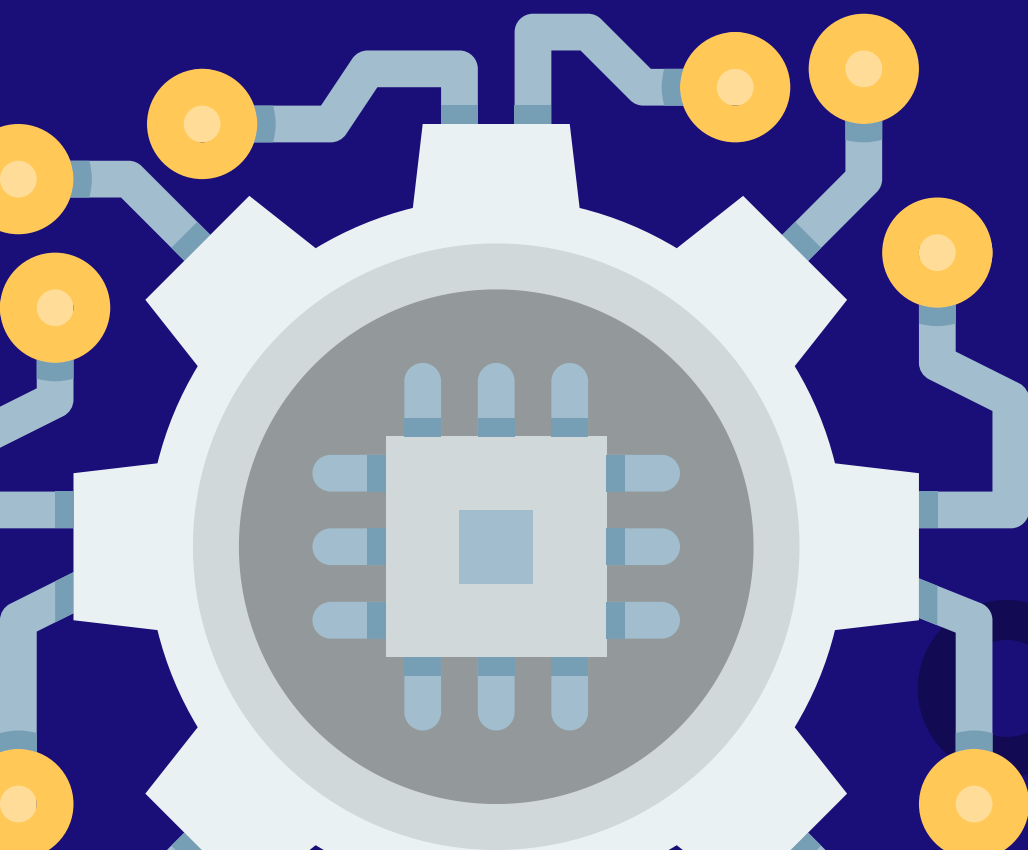
- Wirth estaba preocupado por la dependencia de Pascal respecto a la CDC 6600.
- Él y su grupo de investigación definieron una máquina abstracta.
- Esta máquina utilizaba 0 direcciones.
- El objetivo era hacer que Pascal pudiera ejecutarse en diferentes arquitecturas.

GENERANDO P-CODE

- Wirth modifica un compilador de Pascal. Ahora genera P-Code en lugar de código para la CDC6600.
- Así el P-Code puede ser ejecutado por una P-Machine.

SOFTWARE P-MACHINE

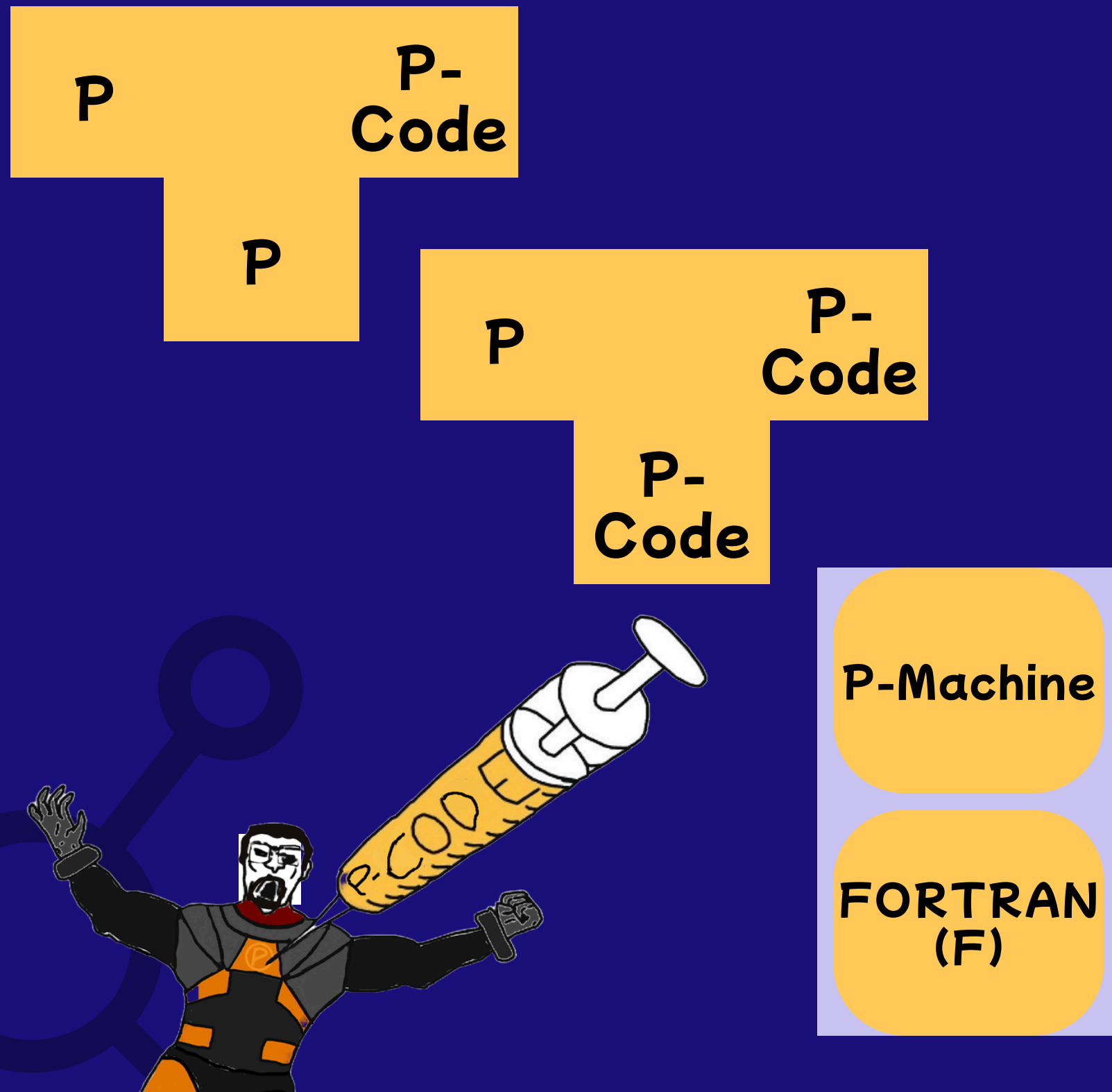
- Se implementa el software de la P-Machine para la CDC 6600.
- El compilador de Pascal puede generar P-Code. La P-Machine interpreta ese P-Code.
- Wirth, completamente fuera de sí, compila el compilador Pascal a P-Code.



Zurich Compiler Kit

Wirth comenzó a distribuir el Compiler Kit a quienes lo solicitaban. El kit incluía herramientas para portar Pascal a nuevas máquinas.

El Compiler Kit incluía:



1 Un compilador Pascal (P), escrito en pascal, que genera P-Code

2 El mismo compilador ya compilado en P-Code

3 Un simulador de P-Machine escrito en Pascal

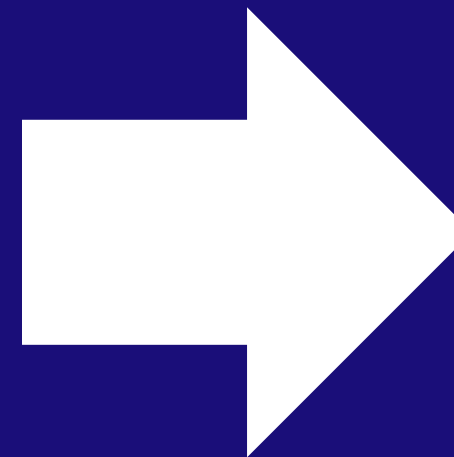
Qué hacemos con todo esto?

Paso 1 - Estudiar la P-Machine

- El intérprete de la P-Machine es pequeño y está escrito en Pascal.
- Podemos estudiarlo y entender cómo funciona. Luego lo reescribimos manualmente en el lenguaje disponible en nuestra máquina.

P-Machine

**FORTTRAN
(F)**



P-Machine

Pascal

Qué hacemos con todo esto?

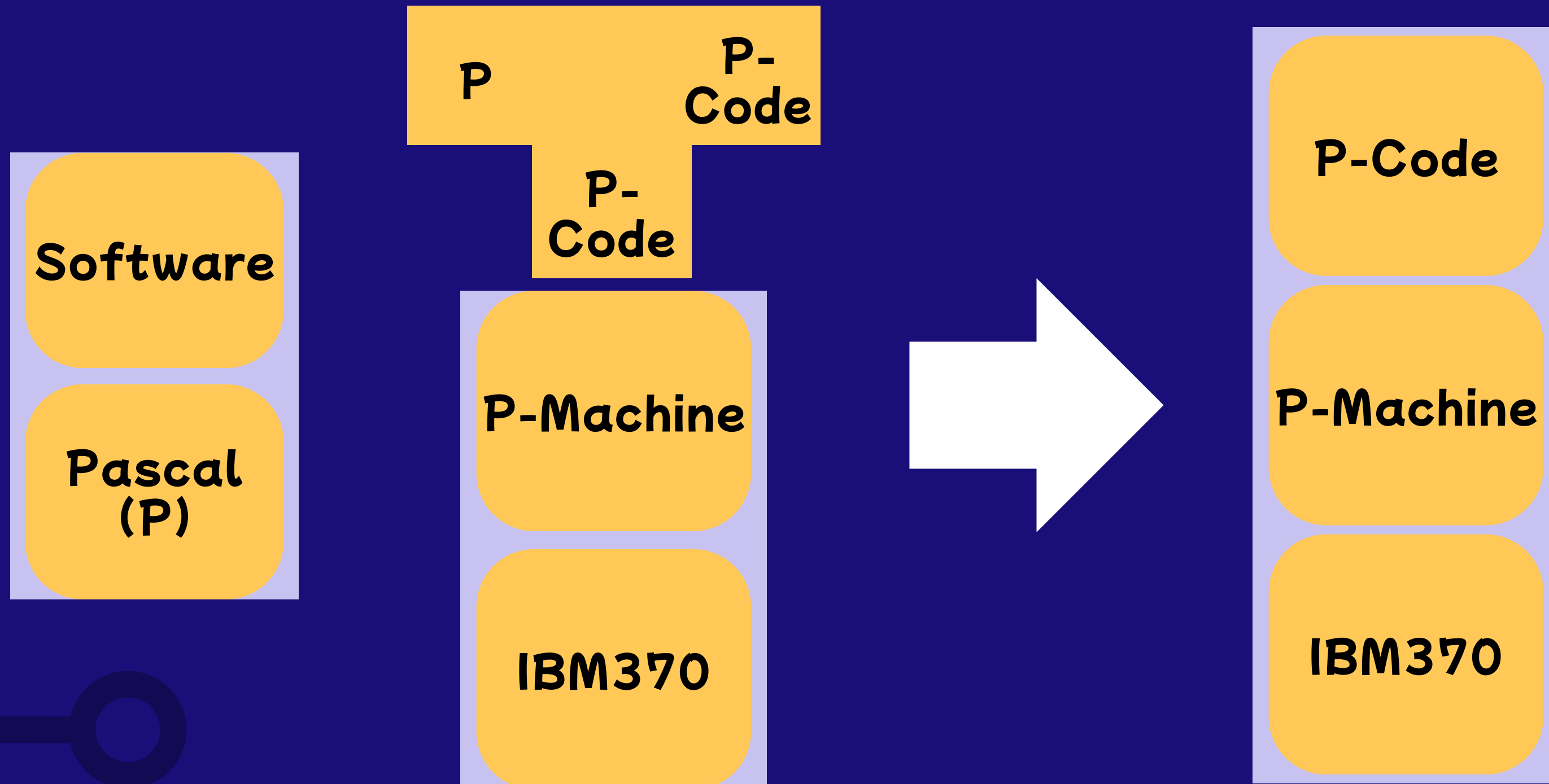
Paso 2 - Estudiar el intérprete

- Tenemos un lenguaje local y un compilador existente.
- Escribimos el intérprete en ese lenguaje.
- Lo compilamos con el compilador local.



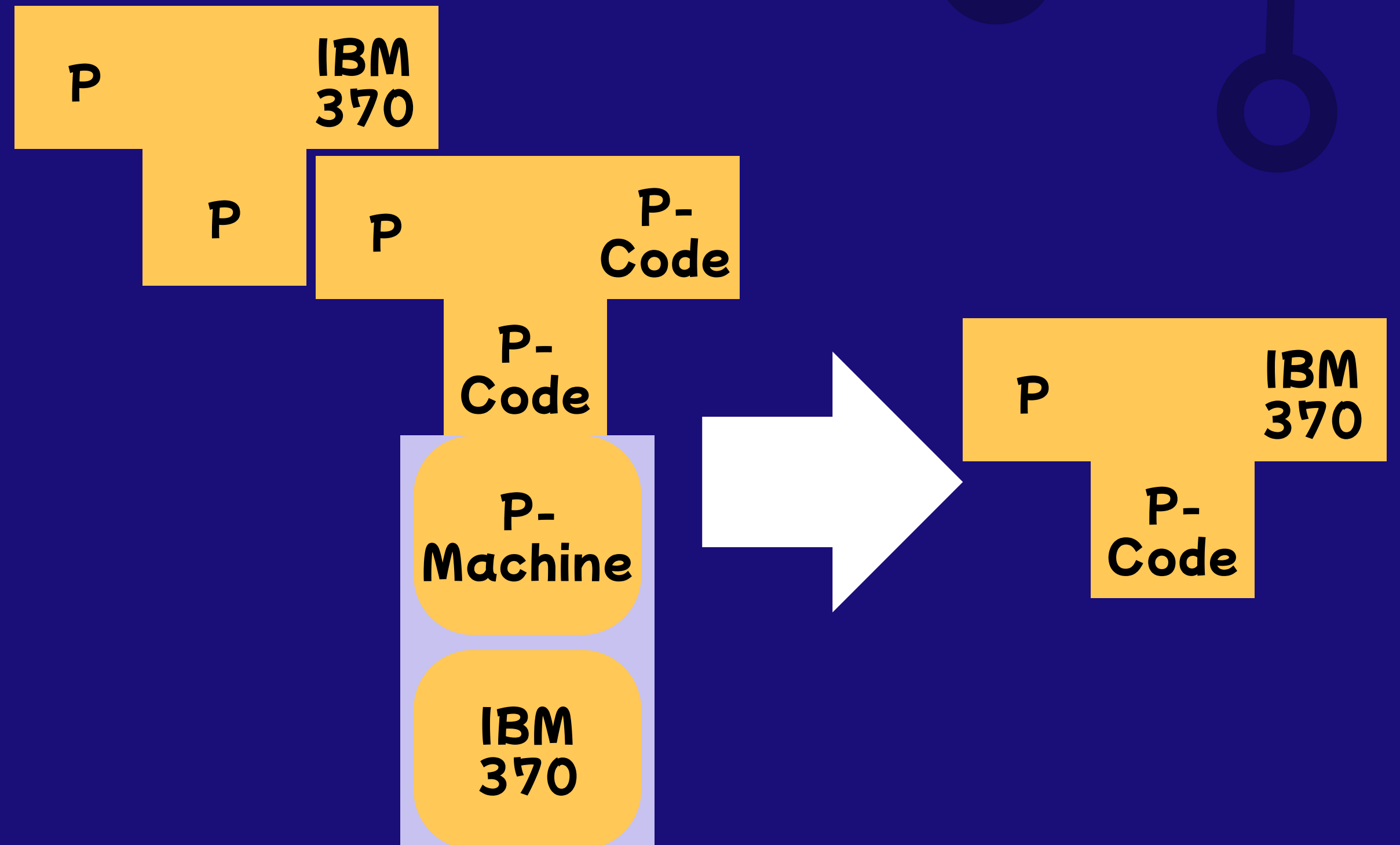
Qué conseguimos?

- Ya tenemos una P-Machine funcional. Podemos ejecutar programas escritos en P-Code.
- El Compiler Kit proporciona un compilador Pascal escrito en P-Code. Podemos ejecutar ese compilador sobre nuestra P-Machine

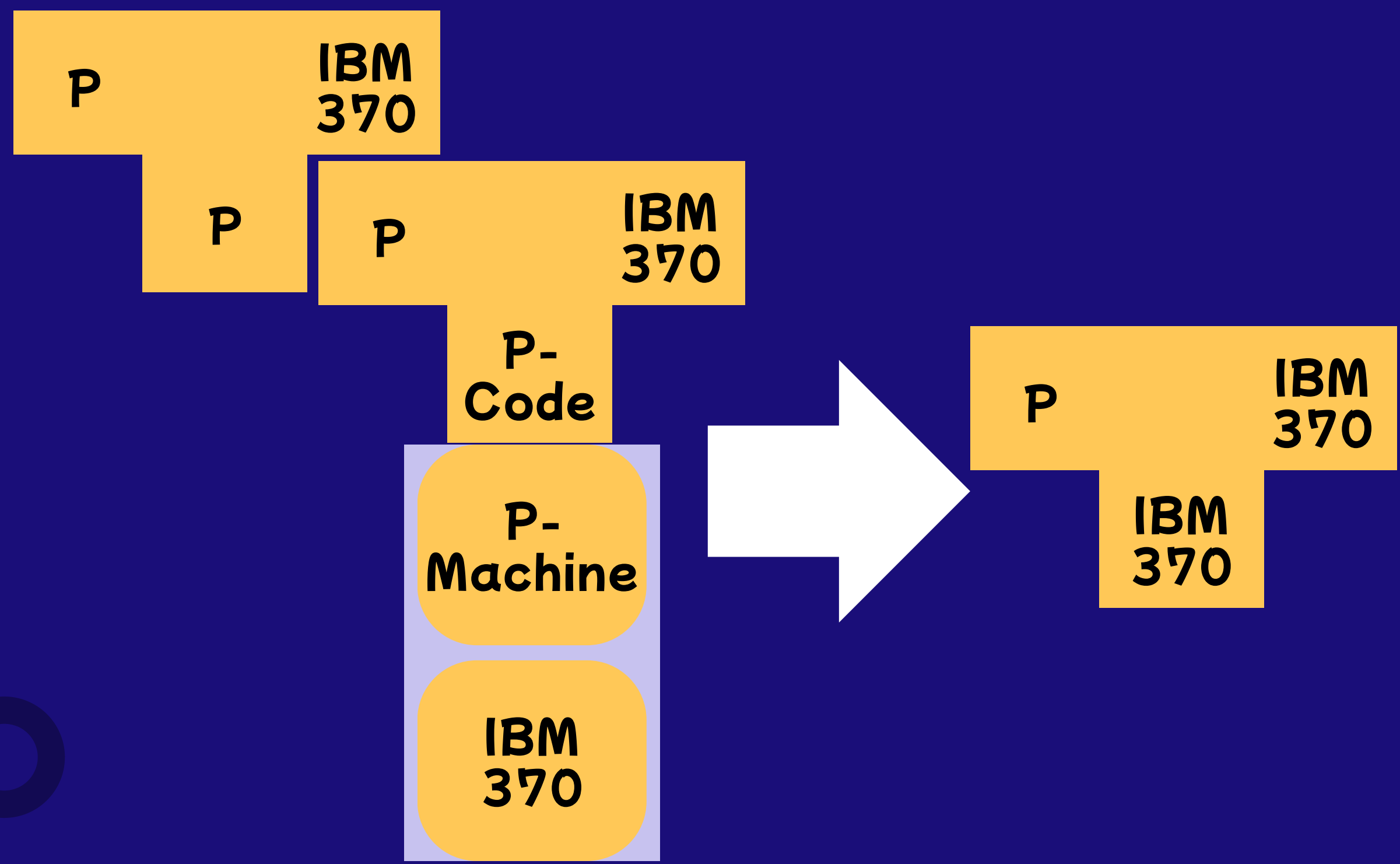


Estudiar compilador Pascal

El compilador Pascal es muy legible. Podemos estudiar cómo funciona y modificarlo manualmente para cambiar la parte encargada de generar código y que produzca lenguaje de nuestra arquitectura disponible

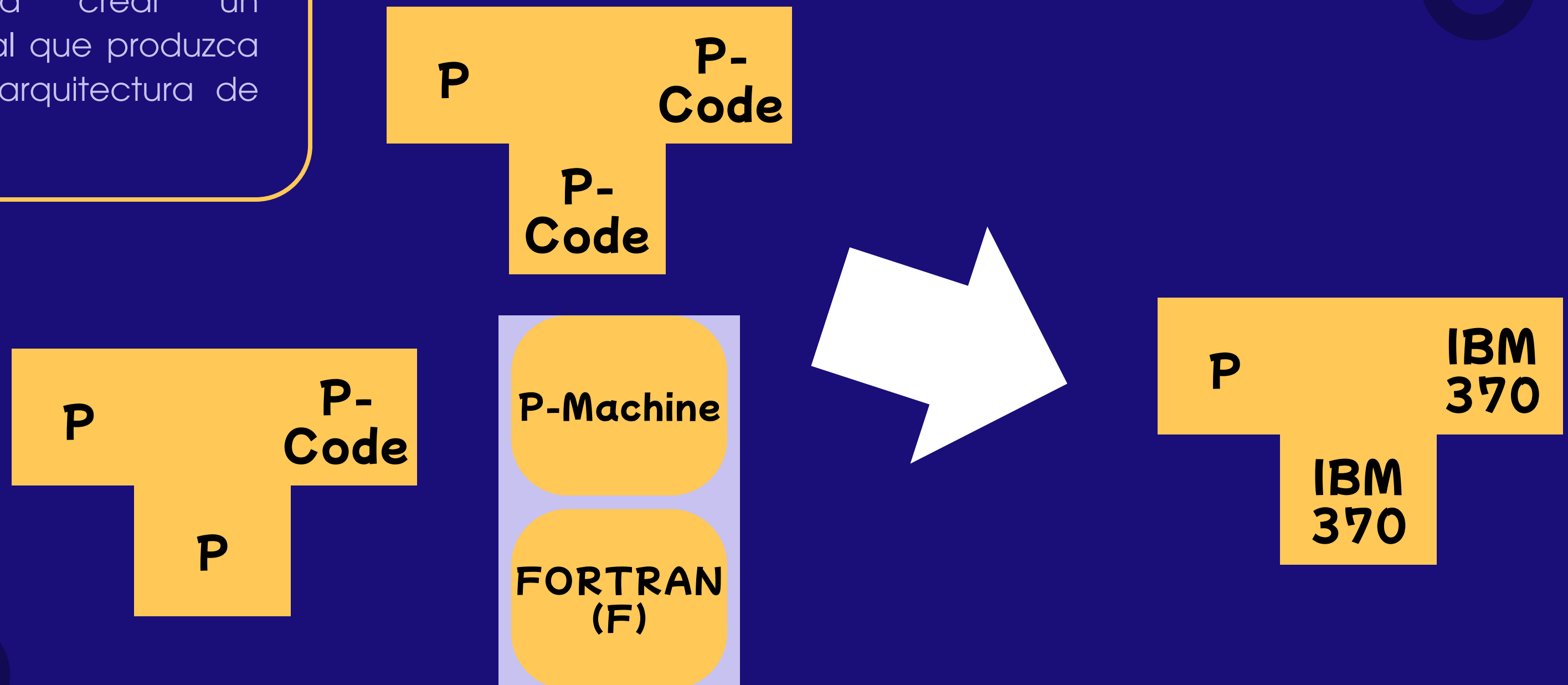


**Compilar nuevo compilador -
Compilador en 370 que genera 370**



Qué hicimos con Zurich Compiler Kit?

Usamos las herramientas disponibles para crear un compilador Pascal que produzca código para la arquitectura de nuestra elección

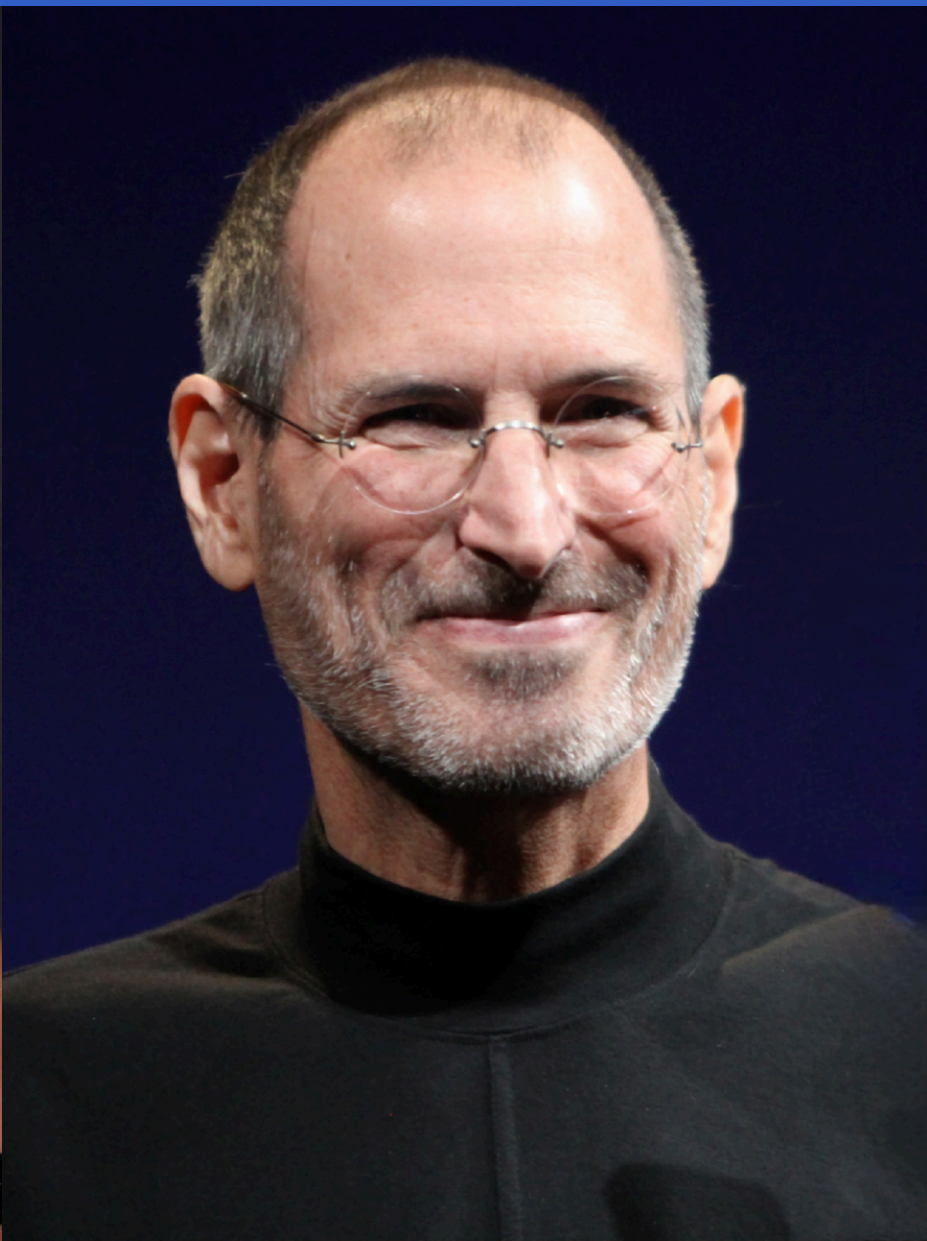


OPINION TORRES

DEL DÍA:

OVERRATED

IDIOTA



EL PROFE

